

## Agenda.

- Dependency Injection
- Middleware
- CDRs
- Background Tasks.

## # Dependency Injection

→ It is a pattern where a function declares what it needs & the framework provides that dependency automatically.

```
fun() {  
  xyz()  
}
```

```
xyz() {  
  ===  
  ===  
  ===  
  ===  
}
```

→ Instead of function creating its own dependencies

Ex - Create a DB connection

- verifying credentials

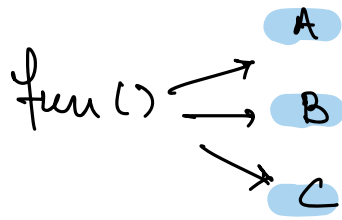
===  
===  
===

other functionalities.

↗

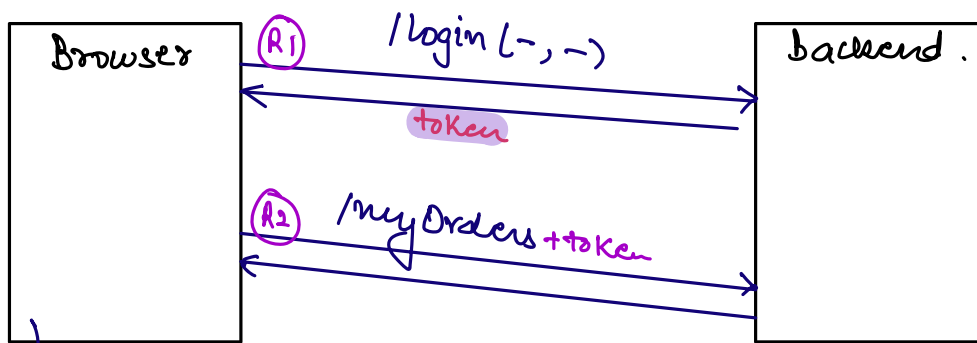
fun just declares all the dependencies it requires & framework will automatically provide these dependencies.

⇒ Depends L) keyword.



# HTTP

→ Every http request is independent.



Amazon.in → —

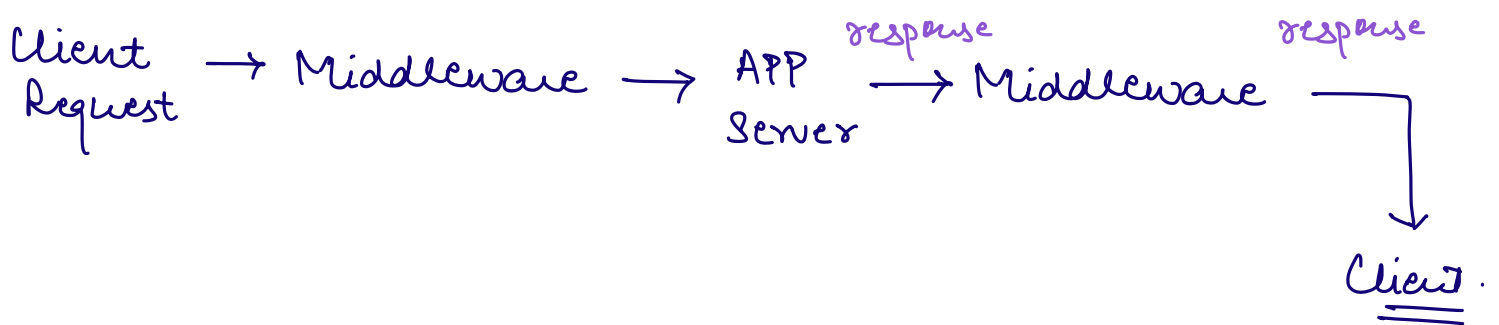
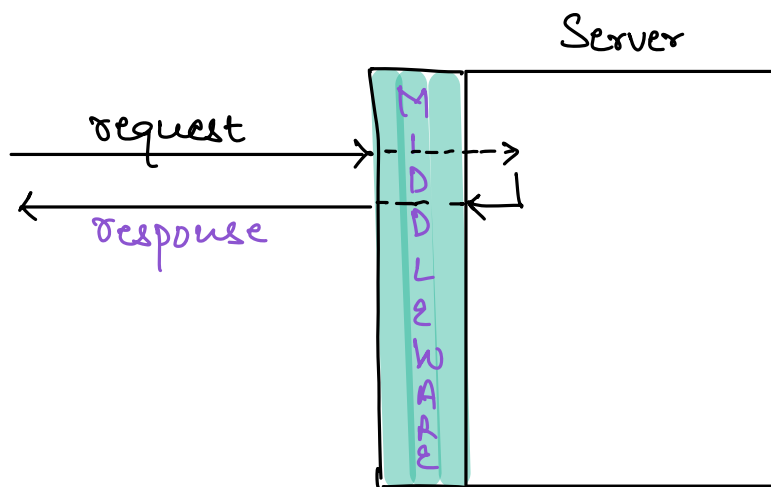
massaischool.com → —

fb.com → —

—————  
—————  
—————  
—————

## # Middleware.

- It is a layer that wraps our entire application.
- Every request passes through middleware before it reaches the actual route/API.
- Every response passes through middleware before it leaves the application.



@app.middleware(http)

Common middleware actions include:

**1. Authentication**

- Verify whether a user is logged in.
- Validate JWT tokens or session data.

**2. Authorization**

- Check user roles and permissions.
- Restrict access to specific routes.

**3. Request Validation**

- Validate headers, query parameters, or request bodies.
- Reject malformed requests.

**4. Logging**

- Record incoming requests and outgoing responses.
- Track performance metrics.

**5. Error Handling**

- Catch exceptions.
- Return standardized error responses.

**6. Request/Response Modification**

- Add or modify headers.
- Transform request or response data.

**7. Redirection**

- Redirect users to another route or URL.
- Force HTTPS.

**8. Rate Limiting**

- Limit the number of requests from a client.
- Prevent abuse or DDoS attacks.

**9. Caching**

- Serve cached responses.
- Improve application performance.



# CORS.

- Cross-Origin Resource Sharing.
- Browser Security Feature.

domain + port.



⇒ Same-Origin policy says a page loaded from one origin can't make requests to a different origin unless that second origin explicitly permits it.

fast API server1 ⇒ 8000  
fast API server2 ⇒ 9000



Application = frontend + Backend.

↓                      ↓  
Port = 3000          Port = 8000.

frontend → Backend



If we want to allow the backend to be accessed via frontend, we'll have to enable CORS on backend server

fastAPI  $\Rightarrow$  CORS Middleware.

